

Bootloader

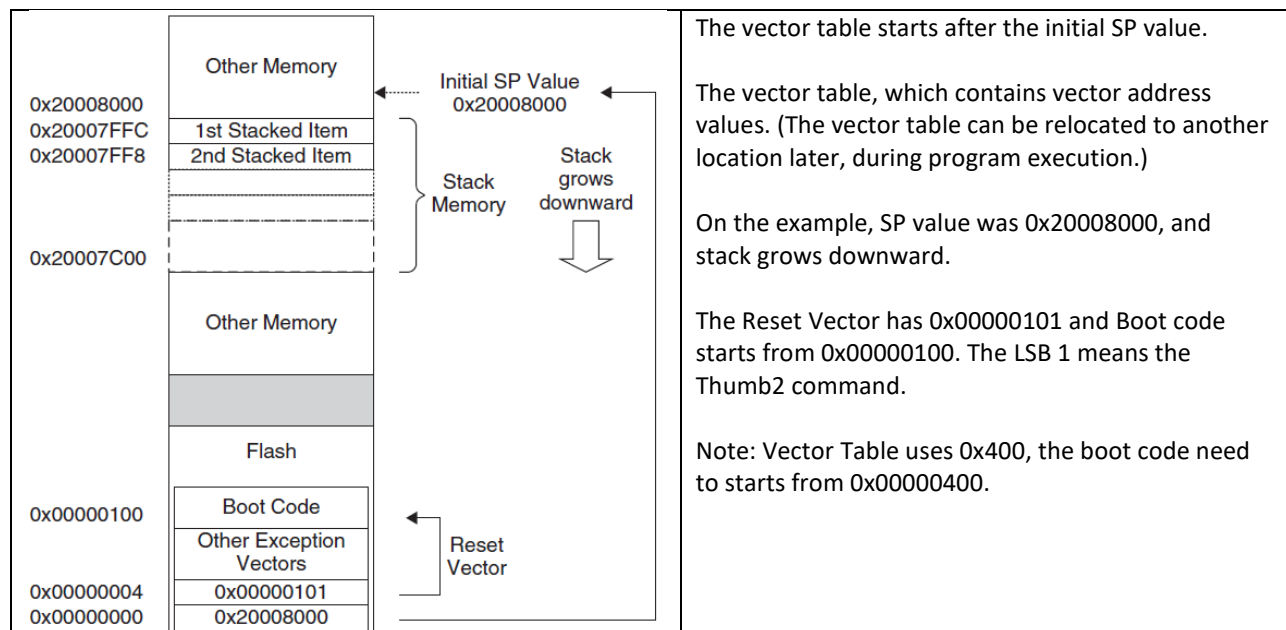
Author: Brian Kang
Last Modified: 5/8/2026

1. Boot Process

After the processor exits reset, it will read two words from memory:

- Address 0x00000000: Starting value of R13 (the stack pointer)
- Address 0x00000004: Reset vector (the starting address of program execution; LSB should be set to 1 to indicate Thumb state) (0:ARM state, 1:Thumb state)

At first set the initial stack pointer value, and then, after the reset vector is fetched, the Cortex-M3 can then start to execute the program from the reset vector address and begin normal operations.



Summary:

Booting Sequence:

1	Read pins BOOT0 and BOOT1 and determine boot mode. (It will be explained later)
2	Fetch MSP form address 0x00000000
3	Fetch PC from address 0x00000004 (LSB need 1 for Thumb)

The value on 0x00000004 indicates below Reset_Handler() address.

The reset handler executes main().

```
void Reset_Handler()
{
```

```

..
main();
..
}

```

C vector table example

```

__attribute__((section(".vector_table"))) unsigned long myvectors[] =
{
    (unsigned long) &_stack_top,
    (unsigned long) rst_handler,
    (unsigned long) nmi_handler,
    // ...
}

```

1.1 bare-metal firmware

1	PC (Program Counter) will be loaded with 0x00000000. So will start from the address 0x00000000.								
2	Since the address has an Initial Stack Pointer value, It will be fetched to the MSP (Main Stack Pointer). So, that value will be the starting of the stack.								
3	Then PC will be loaded with the Reset handler's address. Note: These above 3 steps are done by the hardware (This is architecture-specific).								
4	After that, the reset handler will perform the below operations. <table border="1"> <tr> <td>1</td><td>Initialize the system.</td></tr> <tr> <td>2</td><td>Copy the Initialized global variable, and static variable (.data) to SRAM.</td></tr> <tr> <td>3</td><td>Copy the Un-initialized data (.bss) to SRAM and initialize it to 0.</td></tr> <tr> <td>4</td><td>It Calls main().</td></tr> </table>	1	Initialize the system.	2	Copy the Initialized global variable, and static variable (.data) to SRAM.	3	Copy the Un-initialized data (.bss) to SRAM and initialize it to 0.	4	It Calls main().
1	Initialize the system.								
2	Copy the Initialized global variable, and static variable (.data) to SRAM.								
3	Copy the Un-initialized data (.bss) to SRAM and initialize it to 0.								
4	It Calls main().								

Note: This process is applicable when you don't have a bootloader. If you have a bootloader, then it will execute in a different way.

1.2 With bootloader

The bootloader will be placed in the 0x00000000 with its Vector Table.

The application will be placed in another area of the flash memory with its vector table.

1	When you press the reset button, it will start from the bootloader. So, it will copy the Stack pointer to MSP (Main Stack Pointer) from the location of 0x00000000 .
2	Then it moves to the bootloader's reset handler . It is using the bootloader's vector table .
3	The bootloader will do some operations based on your need like check for firmware version, upgrade the firmware, etc.
4	Once it has done with its operation, it will jump to the application's vector table .
5	Then it will initialize the MSP (Main Stack Pointer) again.
6	Now we have to tell the controller to use the application's vector table instead of using the bootloader's vector table. That will be done using the Vector Table Offset Register (VTOR).
7	Then it goes to the application's reset handler . There it will copy and initialize the data segments to the RAM (Global, Static variables).
8	Finally, it moves to the main function of the application.

2. BootLoader

A **Bootloader** is a small piece of software that runs immediately after you turn on a device. Its primary job is to initialize the hardware and load the operating system into the system's memory (RAM). Think of it as the "bridge" between the hardware and the software.

Bootloader features

Hardware Initialization	It wakes up essential components like the CPU, memory controllers, and storage.
System Integrity	CRC check. In modern systems, it often checks digital signatures to ensure the software hasn't been tampered with (Secure Boot).
Loading the OS	It locates the OS kernel (like Linux or Windows) on a disk, copies it into RAM, and then hands over control of the CPU to that kernel.
Update	Update application (or OS) by UART or OTA

2.1 Boot pin

When you press the reset button, the processor starts running from 0x00000000 address.

Every microcontroller has one or a few boot pins. Using that we can tell the processor to start from the address that we want.

In case of two boot pins

Boot0 pin	register and default value	
0 Ground	BOOT_ADDR0[15:0] 0x00200000	Boot from Custom Bootloader
1 High	BOOT_ADDR1[15:0] 0x00100000	Boot from System Bootloader provided by manufacturer

Both address act as 0x00000000.

In case of two boot pins

Boot1	Boot0	Boot Mode
X	0	Boot from main flash memory
0	1	Boot from system memory (bootloader). "System Bootloader" from system manufacturer
1	1	Boot from embedded SRAM

2.2 Internal FlashMemory

AXIM and ICTM interface bus

	permission	Start Address
ICTM bus	read, executable	0x00200000~
AXIM bus	read, write, execute	0x00800000~

Both address are remapped same physical Flash memory.

Both are remapped to 0x00000000 address.

Table 3. 2 Mbytes Flash memory single bank organization (256 bits read width)

Block	Name	Bloc base address on AXIM interface	Block base address on ICTM interface	Sector size
Main memory block	Sector 0	0x0800 0000 - 0x0800 7FFF	0x0020 0000 - 0x0020 7FFF	32 KB
	Sector 1	0x0800 8000 - 0x0800 FFFF	0x0020 8000 - 0x0020 FFFF	32 KB
	Sector 2	0x0801 0000 - 0x0801 7FFF	0x0021 0000 - 0x0021 7FFF	32 KB
	Sector 3	0x0801 8000 - 0x0801 FFFF	0x0021 8000 - 0x0021 FFFF	32 KB
	Sector 4	0x0802 0000 - 0x0803 FFFF	0x0022 0000 - 0x0023 FFFF	128 KB
	Sector 5	0x0804 0000 - 0x0807 FFFF	0x0024 0000 - 0x0027 FFFF	256 KB
	Sector 6	0x0808 0000 - 0x080B FFFF	0x0028 0000 - 0x002B FFFF	256 KB
	Sector 7	0x080C 0000 - 0x080F FFFF	0x002C 0000 - 0x002F FFFF	256 KB
	Sector 8	0x0810 0000 - 0x0813 FFFF	0x0030 0000 - 0x0033 FFFF	256 KB
	Sector 9	0x0814 0000 - 0x0817 FFFF	0x00340000 - 0x0037 FFFF	256 KB
	Sector 10	0x0818 0000 - 0x081B FFFF	0x0038 0000 - 0x003B FFFF	256 KB
	Sector 11	0x081C 0000 - 0x081F FFFF	0x003C 0000 - 0x003F FFFF	256 KB
Information block	System memory	0x1FF0 0000 - 0x1FF0 EDBF	0x0010 0000 - 0x0010 EDBF	60 Kbytes
	OTP	0x1FF0 F000 - 0x1FF0 F41F	0x0010 F000 - 0x0010 F41F	1024 bytes
	Option bytes	0x1FFF 0000 - 0x1FFF 001F	-	32 bytes

2.2.1 Flash function

HAL_FLASH_Unlock()
 HAL_FLASHx_Erase()
 HAL_FLASH_Program()
 HAL_FLASH_Lock()

2.2.2 Link Script

How to place the code to the boot memory(internal flash, system memory, internal SRAM) ?

A: Linker Script

How are data and code sections merged ?

Where should each section be placed in the memory ?

2.2.3 Set value on 0x0000 and 0x0004

How to set values in memory address 0 and address 4?

A: startup_xxx.s (called as startup code)

Stack_Size	EQU	0x00000400	
	AREA	STACK, NOINIT, READWRITE, ALIGN=3	
Stack_Mem	SPACE	Stack_Size	
__initial_sp			
...			
__Vectors	DCD	__initial_sp	; Top of Stack
	DCD	Reset_Handler	; Reset Handler

startup_stm321xx_md.s

2.3 System Bootloader

The system bootloader in STM32 is the bootloader that has been provided by the chip manufacturers. We cannot overwrite as it has loaded into the ROM memory. We can also call this a **ROM bootloader**. Using this bootloader, we can update the firmware or application from the bootloader.

But we cannot do many operations like updating the application wirelessly like OTA

2.4 Custom Bootloader

Launching the Main Application

step	
1	disable interrupts. C: <code>__disable_irq();</code>
2	swap new vector table. VTOR (Vector table Offset Register) C: <code>SCB->VTOR = APP_ADDRESS;</code>
3	Jumping execution. PC, SP, see <code>boot_jump()</code> on below example

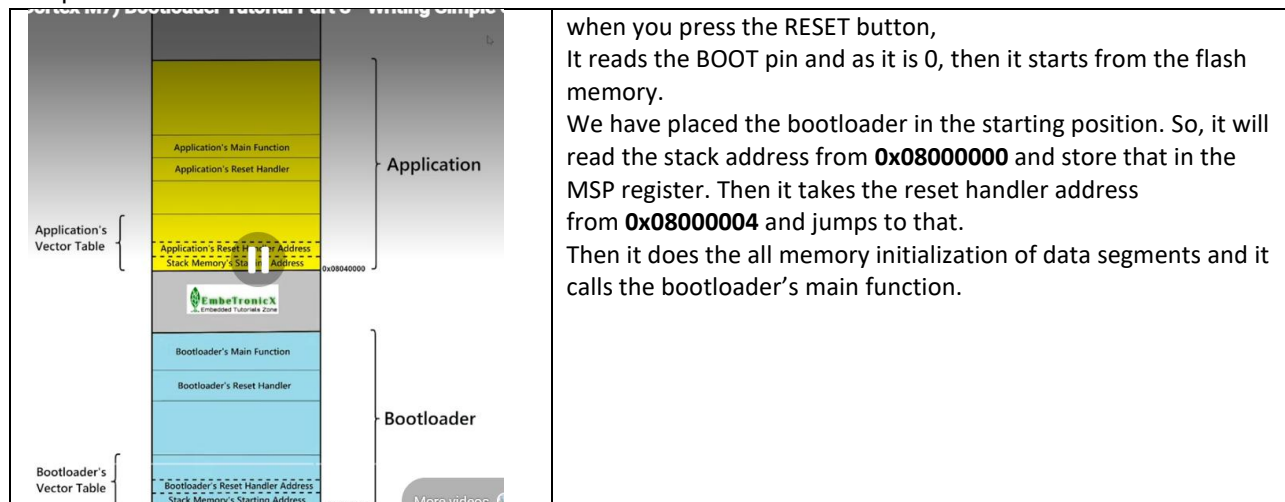
main() example

```
#define APP_ADDRESS 0x08020000

void boot_jump(uint32_t address)
{
    asm volatile("LDR SP, [%0]" :: "r" (address));
    asm volatile("LDR PC, [%0]" :: "r" (address+4));
}

int main()
{
    __disable_irq();
    SCB->VTOR = APP_ADDRESS;
    boot_jump(APP_ADDRESS);
}
```

map

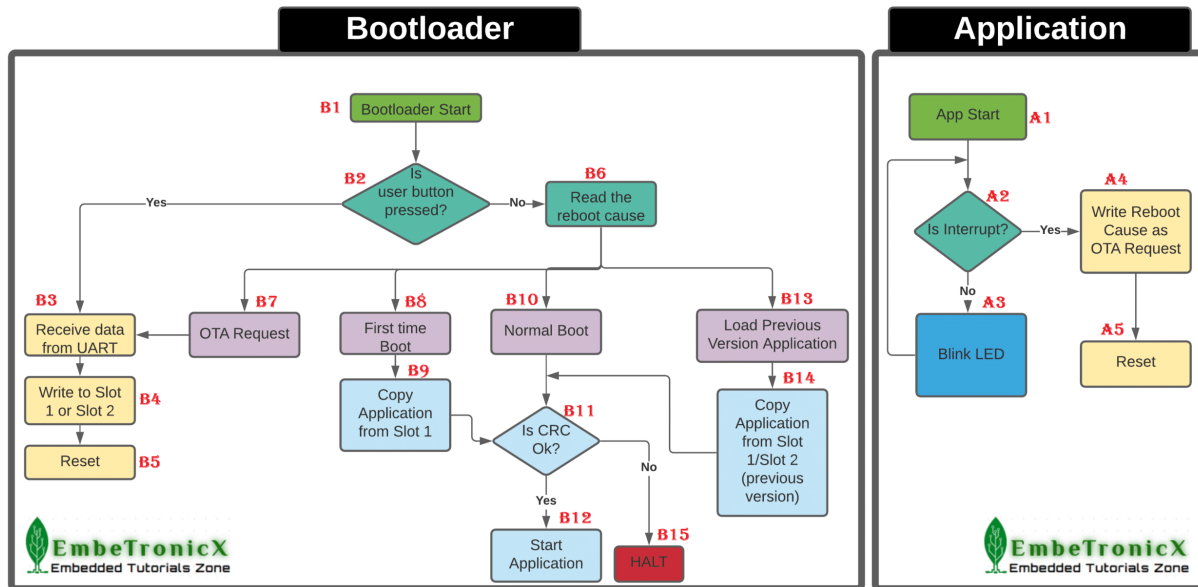


Linker example

	STM32F767ZITX_FLASH.ld example
Bootloader	<pre>/* Memories definition */ MEMORY { RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 512K FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K /* Allocating 64K for Application */ }</pre>
Application	<pre>/* Memories definition */ MEMORY { RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 512K FLASH (rx) : ORIGIN = 0x08040000, LENGTH = 512K /* Allocating 512K for Application */ }</pre>

3 Custom Bootloader Design

Reference: <https://embetronicx.com/stm32f7-bootloader-tutorial/>



Bootloader Process Flow

	Description	Next
B1	The bootloader starts when the microcontroller powers up or resets.	Goto B2
B2	Check the user boot button is pressed. This boot pin is our bootloader pin to identify the Firmware/Application update process.	If Yes, go to Block B3. If No, then go to Block B6.
B3	Receive the data from Host through the UART.	Goto Block B4.
B4	Write the received data to either slot 1 or slot 2 based on availability.	Goto Block B5.
B5	Reset the microcontroller, So that the bootloader runs again. Todo: Update the reboot cause as "First time Boot"	B1
B6	The reboot causes will be stored in a particular area. Read that reboot cause and go to Block B7, Block B8, Block B10, and Block B13.	B7,B8,B10,B13
B7	If the application requests for the OTA update then go to Block B3.	B3
B8	If this is the very first time boot, then go to Block B9.	B11
B9	As this is the very first boot, the bootloader will copy the application from slot 1 to the actual application area which is 0x08040000 . Then go to Block B11.	
B10	If this is the normal boot , then go to Block B11.	B11
B11	Verify the CRC from the actual application location. This will help us to identify whether the application has been modified by some hacker or any memory corruption. If CRC verification is successful then go to Block B12.	If correct CRC, go to B12 if not, go to B15

	If CRC verification is failed then go to Block B15.	
B12	Give control to the application.	A1
B13	If the application requests to load the previous application, then go to Block B14.	B14
B14	Copy the old application from slot 1 or slot 2 to the actual application and go to Block B11.	B11
B15	If the CRC verification is failed, then don't run the application and stay there. Todo: Update the reboot cause as "Load Previous Version"	Halt

Application Process Flow

		Next step
A1	The bootloader starts the application	A2
A2	Check for the GPIO interrupt. If an interrupt occurs, then go to Block A4. Else, go to Block A3.	If IRQ, go A4 else go A3
A3	Blink the LED.	A2
A4	We have received the interrupt, Which means some OTA package is available. So, write the reboot reason as an OTA request. Then go to Block A5.	A5
A5	Reset the device. So that the Bootloader will receive the OTA package and update it.	

Flash memory map (0x08000000~0x081FFFFFF), 2 MB Flash

sector	address	size	Description
11	0x081C0000~	256KB	Reserved
10	0x08180000~	256KB	Slot2
9	0x08140000~	256KB	
8	0x08100000~	256KB	Slot1
7	0x080C0000~	256KB	
6	0x08080000~	256KB	Application
5	0x08040000~	256KB	
4	0x08020000~	128KB	Configuration
3	0x08018000~	32KB	Reserved
2	0x08010000~	32KB	
1	0x08008000~	32KB	Bootloader
0	0x08000000~	32KB	

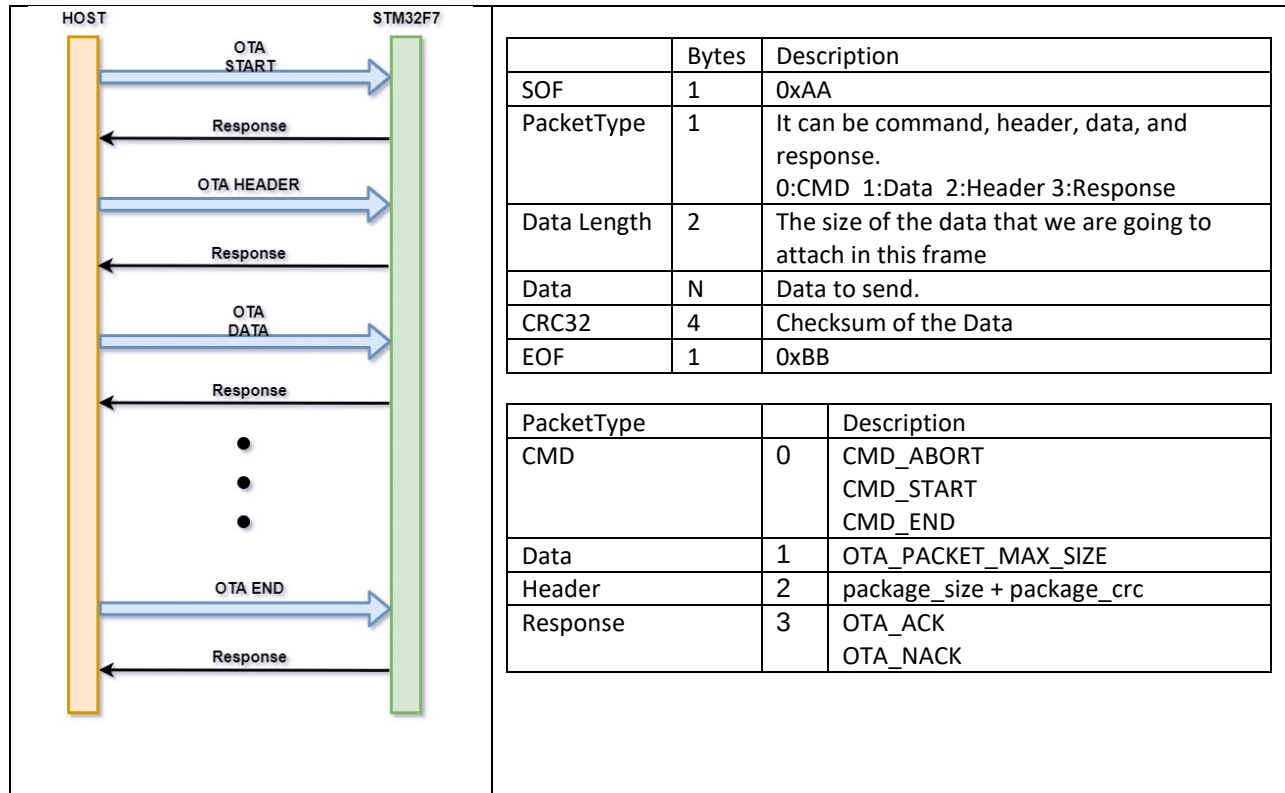
slot1/slot2 used for Backup and Update area.

application sector is for current working application.

3.1 OTA (Over-The-Air) protocol

If you pressed button on B2 step, or OTA routine is requested on B7, B3 OTA routine is executed.

It downloads application firmware from Host to Flash download slot1 or slot2 and then do reset routine.



3.2 Configuration

Read	ETX_GNRL_CFG_ *cfg_flash = (ETX_GNRL_CFG_ *) (ETX_CONFIG_FLASH_ADDR);
Update	ETX_GNRL_CFG_ cfg; memcpy(&cfg, cfg_flash, sizeof(ETX_GNRL_CFG_)); write_cfg_to_flash(&cfg)

<pre> #define ETX_APP_FLASH_ADDR 0x08040000 //Application's Flash Address #define ETX_APP_SLOT0_FLASH_ADDR 0x080C0000 //App slot 0 address #define ETX_APP_SLOT1_FLASH_ADDR 0x08140000 //App slot 1 address #define ETX_CONFIG_FLASH_ADDR 0x08020000 //Configuration's address #define ETX_NO_OF_SLOTS 2 //Number of slots #define ETX_SLOT_MAX_SIZE (512 * 1024) //Each slot size (512KB) #define ETX_OTA_DATA_MAX_SIZE (1024) //Maximum data Size #define ETX_OTA_DATA_OVERHEAD (9) //data overhead #define ETX_OTA_PACKET_MAX_SIZE (ETX_OTA_DATA_MAX_SIZE + ETX_OTA_DATA_OVERHEAD) </pre>	
<pre> /* * Reboot reason */ #define ETX_FIRST_TIME_BOOT (0xFFFFFFFF) //First time boot #define ETX_NORMAL_BOOT (0xBEEFEED) //Normal Boot #define ETX_OTA_REQUEST (0xDEADBEEF) //OTA request by application </pre>	

```
#define ETX_LOAD_PREV_APP    ( 0xFACED )    //App requests to load the previous version
```

```
/*
 * Slot table
 */
typedef struct
{
    uint8_t is_this_slot_not_valid; //Is this slot has a valid firmware/application?
    uint8_t is_this_slot_active;    //Is this slot's firmware is currently running?
    uint8_t should_we_run_this_fw;  //Do we have to run this slot's firmware?
    uint32_t fw_size;                //Slot's firmware/application size
    uint32_t fw_crc;                 //Slot's firmware/application CRC
    uint32_t reserved1;
    uint32_t reserved2;
    uint32_t reserved3;
}__attribute__((packed)) ETX_SLOT;
```

```
/*
 * General configuration
 */
typedef struct
{
    uint32_t reboot_cause;
    ETX_SLOT_slot_table[ETX_NO_OF_SLOTS];
}__attribute__((packed)) ETX_GNRL_CFG;
```

When it download firmware to slot, it uses below function to select the slot_num_to_write.

```
/**
 * @brief Return the available slot number
 * @param none
 * @retval slot number
 */
static uint8_t get_available_slot_number( void )
{
    uint8_t slot_number = 0xFF;

    /* Read the configuration */
    ETX_GNRL_CFG_cfg;
    memcpy( &cfg, cfg_flash, sizeof(ETX_GNRL_CFG_) );
    /*
     * Check the slot is valid or not. If it is valid,
     * then check the slot is active or not.
     *
     * If it is valid and not active, then use that slot.
     * If it is not valid, then use that slot.
     */

    for( uint8_t i = 0; i < ETX_NO_OF_SLOTS; i++ )
    {
        if( ( cfg.slot_table[i].is_this_slot_not_valid != 0u ) /* it is not valid */
        || ( cfg.slot_table[i].is_this_slot_active == 0u ) ) /* it is not active */
        {
```

```

    slot_number = i;
    printf("Slot %d is available for OTA update\r\n", slot_number);
    break;
}
}

return slot_number;
}

```

When it start writing downloaded data on slot,

```

    cfg.slot_table[slot_num_to_write].is_this_slot_not_valid = 1;

```

When it has OTA_CMD_END,

```

    cfg.slot_table[slot_num_to_write].fw_crc          = cal_crc;
    cfg.slot_table[slot_num_to_write].fw_size         = ota_fw_total_size;
    cfg.slot_table[slot_num_to_write].is_this_slot_not_valid = 0u;          /* now, it is valid */
    cfg.slot_table[slot_num_to_write].should_we_run_this_fw = 1u;          /* */

```

```

    cfg.slot_table[i].should_we_run_this_fw = 0u; /* change the other slot as 0 */

```

When it updating application sector,

```

    if( cfg.slot_table[i].should_we_run_this_fw == 1u ) {
        is_update_available      = true;
        slot_num                  = i;
        /* update status */
        cfg.slot_table[i].is_this_slot_active = 1u;
        cfg.slot_table[i].should_we_run_this_fw = 0u;
    }

```

//make other slots as inactive

```

    cfg.slot_table[i].is_this_slot_active = 0u;

```

If it does not need to update application sector, choose active slot to check CRC,

```

    if( cfg.slot_table[i].is_this_slot_active == 1u )
        slot_num = i

```

use the application sector to generates CRC, and use the CRC information on the selected slot!

When it need to copy firmware from slot to firmware sector on load_new_app(), it checks should_we_run_this_fs.

```

void load_new_app( void )
{
    ..
    /* Read the configuration */
    ETX_GNRL_CFG_cfg;
    memcpy( &cfg, cfg_flash, sizeof(ETX_GNRL_CFG_) );

    /* * Check the slot whether it has a new application. */

    for( uint8_t i = 0; i < ETX_NO_OF_SLOTS; i++ )
    {
        if( cfg.slot_table[i].should_we_run_this_fw == 1u )

```

```

{
    printf("New Application is available in the slot %d!!!\r\n", i);
    is_update_available      = true;
    slot_num                  = i;

    //update the slot
    cfg.slot_table[i].is_this_slot_active  = 1u;
    cfg.slot_table[i].should_we_run_this_fw = 0u;

    break;
}
}

if( is_update_available ) {
    ..
    ret = write_data_to_flash_app( (uint8_t*)slot_addr, cfg.slot_table[slot_num].fw_size );
}
..
}

```

3.3 load new app

load_new_app() does follow two steps

step1	load from slot to Application sector if it has new application or need to use the previous version.
step2	Check CRC before execute the application

3.4 Jump to application

```

/**
 * @brief Jump to application from the Bootloader
 * @retval None
 */
static void goto_application(void)
{
    printf("Gonna Jump to Application\r\n");
    void (*app_reset_handler)(void) = (void*)((volatile uint32_t*) (ETX_APP_FLASH_ADDR + 4U));
    //__set_MSP(*(volatile uint32_t*) ETX_APP_FLASH_ADDR);
    // Turn OFF the Green Led to tell the user that Bootloader is not running
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET ); //Green LED OFF

    /* Reset the Clock */
    HAL_RCC_DeInit();
    HAL_DeInit();
    __set_MSP(*(volatile uint32_t*) ETX_APP_FLASH_ADDR);
    SysTick->CTRL = 0;
    SysTick->LOAD = 0;
    SysTick->VAL = 0;

    /* Jump to application */
    app_reset_handler(); //call the app reset handler
}

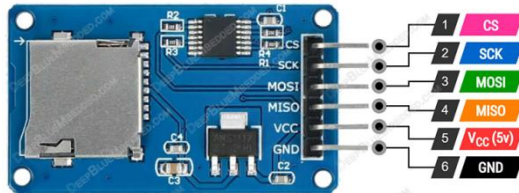
```

4 Other OTA

4.1 Use BT-05 Bluetooth to Serial UART



4.2 SD card reader



<https://01001000.xyz/2020-08-09-Tutorial-STM32CubeIDE-SD-card/>
<https://deepbluembedded.com/stm32-sd-card-spi-fatfs-tutorial-examples/>

Connect SD card reader with SPI1

Copy the firmware file to that SD card.

Then we read that firmware file and update the firmware to the flash memory.

```
f_mount(&FatFs, "", 1); /* mount SD memory on SPI1 */
/* open "/ETX_Fw/app.bin" file */
fres = f_open(&fil, ETX_SD_CARD_FW_PATH, FA_WRITE | FA_READ | FA_OPEN_EXISTING);
UINT fw_size = f_size(&fil);

for(uint32_t i = 0; i < fw_size; ) {
    if( (fw_size - i) >= ETX_OTA_DATA_MAX_SIZE )
        size = ETX_OTA_DATA_MAX_SIZE;
    else
        size = fw_size - i;

    memset(readBuf, 0, ETX_OTA_DATA_MAX_SIZE);
    fres = f_read(&fil, readBuf, size, &bytesRead);
    write_data_to_slot(..)
}
f_close(&fil);
f_mount(NULL, "", 0);
```

5 Secure Bootloader

<https://hubble.com/community/guides/how-to-implement-secure-boot-on-arm-cortex-m/>

The Root of Trust (RoT) is a small, immutable code region: either mask ROM burned into silicon or a write-protected flash sector that nothing on the device can modify at runtime. This code does exactly one thing: verify the next stage.

5.1 Flash Memory Layout

0x0800_0000	Immutable Bootloader (Root of Trust code)	← Write-protected flash sector(s)
0x0800_4000	Public Key Storage (ECDSA P-256 public key)	← OTP fuses or write-protected flash
0x0800_4100	Rollback Counter	← OTP or protected flash
0x0800_4200	Firmware Header	← Magic, version, size, signature
0x0800_4280	Application Firmware	← Verified before execution
0x080X_XXXX		

5.2 Step-by-Step Bootloader Verification

step1	Minimal Hardware Initialization <pre>void bootloader_entry(void) { // Initialize only what's necessary: clocks, flash wait states system_clock_init(); flash_set_wait_states(); // SECURITY NOTE: Do NOT initialize UART, SPI, USB, or any // peripheral. Every active peripheral is attack surface. // No interrupts enabled. Polling only. boot_verify_and_jump(); }</pre>
step2	Read and Validate the Firmware Header <pre>typedef struct { uint32_t magic; // e.g., 0x53424F4F ("SB00") uint32_t version; // Monotonic firmware version uint32_t image_size; // Bytes, not including header uint32_t reserved; uint8_t signature[64]; // ECDSA P-256 signature (r s) } firmware_header_t; int validate_header(const firmware_header_t *hdr) { if (hdr->magic != EXPECTED_MAGIC) return ERROR_INVALID_HEADER;</pre>

	<pre> // SECURITY NOTE: Bounds-check image_size against physical flash. // An unchecked size field lets an attacker control what gets hashed. if (hdr->image_size == 0 hdr->image_size > MAX_APP_SIZE) return ERROR_INVALID_SIZE; // Check rollback counter uint32_t stored_version = read_rollback_counter(); if (hdr->version < stored_version) return ERROR_ROLLBACK_DETECTED; return SUCCESS; } </pre>
step3	Compute the Hash of the Firmware Image <pre> int compute_firmware_hash(const uint8_t *fw_start, uint32_t fw_size, uint8_t *hash_out) { sha256_context ctx; sha256_init(&ctx); // SECURITY NOTE: Hash in chunks to limit stack usage. // On an M0 with 4 KB RAM, a 512-byte chunk is reasonable. uint32_t offset = 0; while (offset < fw_size) { uint32_t chunk = MIN(HASH_CHUNK_SIZE, fw_size - offset); sha256_update(&ctx, fw_start + offset, chunk); offset += chunk; } sha256_finish(&ctx, hash_out); // 32-byte digest return SUCCESS; } </pre>
step4	Verify the Signature <pre> int verify_firmware_signature(const uint8_t *hash, const uint8_t *signature) { // Load public key from protected storage (OTP or write-protected flash) const uint8_t *public_key = (const uint8_t *)PUBLIC_KEY_ADDRESS; int result = ecdsa_verify_p256(public_key, hash, 32, signature, 64); // SECURITY NOTE: Double-check the result to resist single-glitch // fault injection that flips a return value. int result2 = ecdsa_verify_p256(public_key, hash, 32, signature, 64); if (result != ECDSA_VALID result2 != ECDSA_VALID) return ERROR_SIGNATURE_INVALID; return SUCCESS; } </pre>
step5	Enforce the Decision <pre> void boot_verify_and_jump(void) { const firmware_header_t *hdr = (firmware_header_t *)FW_HEADER_ADDRESS; const uint8_t *app_start = (uint8_t *) (FW_HEADER_ADDRESS + sizeof(*hdr)); uint8_t hash[32]; if (validate_header(hdr) != SUCCESS) boot_failure_handler(); // Does NOT return if (compute_firmware_hash(app_start, hdr->image_size, hash) != SUCCESS) </pre>

	<pre> boot_failure_handler(); if (verify_firmware_signature(hash, hdr->signature) != SUCCESS) boot_failure_handler(); // Update rollback counter if firmware version is newer if (hdr->version > read_rollback_counter()) write_rollback_counter(hdr->version); // SECURITY NOTE: jump_to_application must never be reachable // without passing all three checks above. jump_to_application((uint32_t)app_start); } _Noreturn void boot_failure_handler(void) { // Safe state: infinite loop, or signal error via LED/GPIO // SECURITY NOTE: NEVER fall through to application. while (1) { __WFI(); // Low-power wait; device is bricked until re-flashed } } </pre>
step6	Lock Down Before Jumping <pre> void jump_to_application(uint32_t app_address) { // Configure MPU to write-protect bootloader region (if MPU available) mpu_configure_bootloader_readonly(); // Set vector table offset to application's vector table SCB->VTOR = app_address; // Read initial stack pointer and reset handler from app vector table uint32_t app_sp = *(volatile uint32_t *)(app_address); uint32_t app_reset = *(volatile uint32_t *)(app_address + 4); __set_MSP(app_sp); // Jump to application reset handler void (*app_entry)(void) = (void (*)(void))app_reset; app_entry(); } </pre>

5.3 Arm TrustZone STM32

ref: STM AN5447.

5.4 Cortex-M Booting to main()

ref: https://www.linkedin.com/posts/techdhaba_embeddedsystems-arm-cortexm-ugcPost-7413697582184722432-H7mp/

□ Power On → Reset → main()

In reality, ARM Cortex-M devices power-up through a deterministic trust pipeline — a carefully engineered sequence of silicon logic, Boot ROM, startup assembly, memory relocation, and execution hand-off that runs before your C code ever begins.

Your firmware isn't the first thing that runs.

The system ensures that stack, memory, interrupt vectors, and security boundaries are valid — before you get control.

Here's what's really happening inside the MCU.

Phase-1	Power-On Reset & The Hardware Fetch • Power rails stabilize • clocks lock • reset is released The CPU performs two automatic reads at 0x00000000: 1. Loads Main Stack Pointer (MSP) 2. Loads Reset Vector (address of first instruction) The trust boundary is established.
Phase-2	Boot ROM (The Silicon Root of Trust) Immutable vendor firmware: ✓ selects boot source (Flash / UART / USB / CAN) ✓ validates firmware integrity ✓ enforces secure boot / read-out protection Unsigned images simply do not execute. Security happens before software.
Phase-3	User Bootloader (Where Real Systems Differ) Production firmware rarely jumps straight to the app. Bootloader responsibilities: ✓ CRC & integrity verification ✓ rollback safety ✓ firmware updates / DFU ✓ VTOR relocation ✓ load application stack pointer Without this step — the first interrupt can crash the system. Bootloaders don't just "start the app". They transfer system ownership.
Phase-4	Startup Assembly (Reset_Handler) We are still not in C. Startup code: ✓ copies .data Flash → RAM ✓ zero-initializes .bss ✓ configures PLL & clocks ✓ enables FPU ✓ runs C++ constructors Only after the environment is correct... main() is allowed to execute
Phase-5	Application Runtime Now the system becomes your design: □ Bare-metal super-loop □ RTOS scheduler □ Interrupt-driven runtime Stack aligned. Memory valid. Timing deterministic. Your MCU is not just "running". It is reliably alive.

Linux Boot Level

In the world of ARMv8 (AArch64) architecture, these terms refer to the specific stages of the **Trusted Board Boot Requirements (TBBR)**, which is the standard sequence for a **Secure Boot** process. This sequence ensures that every piece of software loaded onto a system is verified and trusted.

Each stage acts as a "link in the chain," initializing hardware and passing control to the next level of firmware.

	Description
BL1	Trusted ROM (Bootloader stage 1) BL1 is the very first code that executes on the CPU after a power-on reset. It is typically burned into the Read-Only Memory (ROM) of the SoC during manufacturing. Primary Job: Initialize the platform and verify the next stage (BL2). Key Tasks: - Sets up the basic architectural state (e.g., Exception Level 3). - Initializes a small amount of internal SRAM for the next stage. - Locates and loads the BL2 image from non-volatile storage (like eMMC or SD card). - Security: It uses a public key stored in the SoC to verify the digital signature of BL2. If the signature is invalid, the boot process stops to prevent unauthorized code from running.
BL2	Trusted Boot Firmware (Bootloader Stage 2) BL2 is usually the first stage of "pluggable" firmware, meaning it resides in flash memory rather than ROM. It runs in Secure EL1. Primary Job: Initialize main memory (DDR) and act as the "dispatcher" for all subsequent stages. Key Tasks: - DRAM Initialization: This is the most critical hardware task, as later stages are often too large for internal SRAM. - Loading Multiple Images: BL2 is responsible for loading BL31, BL32 (Secure OS like OP-TEE, if used), and BL33 (Non-secure bootloader like U-Boot). - Verification: It verifies the signatures of all BL3x images before handing over control.
BL3	EL3 Runtime Software BL31 is a unique and permanent resident in the system. While BL1 and BL2 "disappear" once they finish their jobs, BL31 stays in memory while the operating system is running. Primary Job: Handle the transitions between the "Secure World" and the "Normal World." Key Tasks: - SMC (Secure Monitor Call) Handling: It acts as the gatekeeper. When the OS (Non-secure) needs to perform a secure operation—like turning on a CPU core or accessing a secure variable—it calls BL31. - Power Management: It often implements the PSCI (Power State Coordination Interface), which the Linux kernel uses to manage CPU idle states and hotplugging. - Context Switching: It saves and restores CPU registers when switching the processor between Secure and Non-secure states.

OP-TEE

(Open Portable Trusted Execution Environment) is an open-source Trusted OS designed to run in the "Secure World" of ARM processors using **TrustZone** technology.

It creates an isolated environment that runs alongside a "Normal World" operating system (like Linux) to protect sensitive data and perform secure operations like cryptographic key management or digital rights management (DRM).

1. The Architectural Split: TrustZone

OP-TEE relies on the hardware-based isolation provided by ARM TrustZone. The processor is logically divided into two states:

Rich Execution Environment (REE): The "Normal World" where Linux/Android, user applications, and standard drivers reside.

Trusted Execution Environment (TEE): The "Secure World" where OP-TEE and **Trusted Applications (TAs)** reside.

2. How the Two Worlds Communicate

Because the Secure World is isolated, the Linux kernel cannot directly access OP-TEE's memory. Instead, they communicate through a strictly defined path:

Client Application (CA): A standard program in Linux wants to perform a secure task (e.g., decrypt a file). It uses the **TEE Client API**.

TEE Driver: The request goes to the Linux Kernel's TEE driver.

SMC (Secure Monitor Call): The driver issues a special instruction called SMC. This triggers a "World Switch."

Secure Monitor (BL31): The Secure Monitor (usually part of the firmware like TF-A) intercepts the SMC, saves the Linux state, and switches the CPU to the Secure World.

OP-TEE Core: The request is handed to the OP-TEE kernel.

3. The Execution Flow inside OP-TEE

Once the request reaches the Secure World:

Trusted Application (TA): OP-TEE identifies the specific TA needed (e.g., a "Crypto TA"). If the TA isn't already in secure memory, OP-TEE can load it from the Linux file system (after verifying its digital signature).

Isolation: The TA performs the operation using secure memory that the Normal World cannot see.

Return: Once finished, the result is placed in a shared memory buffer, and another SMC is used to switch back to the Normal World.

4. Key Components of OP-TEE

OP-TEE Core: The microkernel that manages secure resources, scheduling, and memory.

Trusted Applications (TAs): Specialized plugins that run inside the TEE. They are often signed with a private key to ensure only authorized code runs.

Tee-supplciant: A background daemon running in Linux that helps OP-TEE access "Normal World" resources, like loading TA files from the disk or accessing a database.

Why use it?

By using OP-TEE, even if an attacker gains **root access** to your Linux OS, they still cannot access the encryption keys or biometric data stored inside the OP-TEE Secure World. The hardware enforces a "wall" that software cannot bypass without the proper secure-monitor handshaking.

R-CAR IPL(Initial Program Loader)

Loader is on QSPI memory.

IPL download files on Hyper Flash memory.

The "IPL" (Initial Program Loader) files are part of the **Renesas Trusted Firmware-A (TF-A)** implementation.

These files are typically flashed into the **HyperFlash** or **QSPI Flash** and are executed in a specific order to bring the SoC from a "cold" state to a running OS.

fname & location	Description
bootparam_sa0.srec 0x000000	Boot Parameters This is not executable code, but a data structure located at the very beginning of the flash memory (Sector 0). Role: The R-Car internal Boot ROM reads this file first. Content: It contains metadata about the flash layout, including the load addresses and sizes for the subsequent boot stages (like BL2). It tells the hardware "where to find the code."
bl2-h3ulcb.srec 0x040000	Trusted Boot Firmware This corresponds to the BL2 stage we discussed earlier. While the file name says h3ulcb, it is commonly used for the H3SK as they share a similar architecture (ULCB vs. SK). Role: This is the first "pluggable" code loaded into internal SRAM by the Boot ROM. Key Tasks: - Initializes the LPDDR4 RAM. Without this, the system cannot load larger files like U-Boot or Linux. - Initializes the system clocks and basic hardware interfaces (I2C, GPIO). - Loads and verifies the next stages (BL31 and U-Boot).
cert_header_sa6.srec 0x180000	Certificate Headers In a secure boot configuration, you will see several "cert" or "header" files. Role: These contain the digital signatures and public key information required to verify the integrity of the BL2 and BL3x images. Even if you aren't using "Secure Boot" in a development environment, the R-Car bootloader often expects these headers to be present in the flash layout to identify the image boundaries.
bl31-h3ulcb.srec 0x1C0000	EL3 Runtime Software This is the ARM Trusted Firmware (TF-A) Stage 3-1. Role: It manages the Secure Monitor. It stays resident in memory to handle power management (PSCI) and switches between the Secure World (where OP-TEE lives) and the Normal World (where Linux lives).
tee-h3ulcb.srec 0x 200000	OP-TEE OS <i>(Optional, but often present in the IPL suite)</i> Role: This is the binary for the OP-TEE operating system. If your system requires a Trusted Execution Environment, BL2 will load this into a protected region of memory.
u-boot-elf.srec 0x640000	The Bootloader / BL33 This is the final stage of the firmware before the Linux kernel. Role: U-Boot provides the user interface (the command prompt over UART) and the logic to load the Linux kernel from an SD card, eMMC, or over a network (TFTP). Format: The .srec (Motorola S-record) format is used here specifically so that the Renesas Flash Writer tool can easily parse the memory addresses and write the data to the correct location in the serial flash.

U-Boot Parameters

	Parameters
WS1.1 or WS2.0	<pre>=> setenv bootargs_nfs 'rw root=/dev/nfs rootwait nfsroot=192.168.1.2:/srv/nfs/50900-h3-weston,vers=3 ip=dhcp' => setenv bootcmd_nfs 'setenv bootargs \${bootargs_nfs};bootp 0x48080000 Image;tftp 0x48000000 \${dtb};run booti_cmd' => setenv booti_cmd 'booti 0x48080000 - 0x48000000' => setenv dtb 'r8a77951-h3ulcb.dtb' => setenv ethact 'ravb' => setenv ethaddr '2e:09:0A:01:99:53' => setenv serverip '192.168.1.2'</pre>

	<pre>=> setenv ipaddr '192.168.1.7' => saveenv => run bootcmd_nfs</pre>
WS3.0	<pre>=> setenv bootargs_nfs 'rw root=/dev/nfs rootwait nfsroot=192.168.1.2:/srv/nfs/50900-h3-weston,vers=3 ip=dhcp' => setenv bootcmd_nfs 'setenv bootargs \${bootargs_nfs};bootp 0x48080000 Image;tftp 0x48000000 \${dtb};run booti_cmd' => setenv booti_cmd 'booti 0x48080000 - 0x48000000' => setenv dtb 'r8a779m1-ulcb.dtb' => setenv ethact 'ravb' => setenv ethaddr '2e:09:0A:01:99:53' => setenv serverip '192.168.1.2' => setenv ipaddr '192.168.1.7' => saveenv => run bootcmd_nfs</pre>
SD	<pre>=> setenv bootargs_sd0 'rw root=/dev/mmcblk1p1 rootfstype=ext4 rootwait' => setenv bootcmd 'run bootcmd_sd0' => setenv bootcmd_sd0 'setenv bootargs \${bootargs_sd0};ext2load mmc 0:1 0x48080000 /boot/Image;ext2load mmc 0:1 0x48000000 /boot/\${dtb};run booti_cmd' => saveenv => run bootcmd</pre>
VBox	<pre>=> setenv bootargs_vb 'rw root=/dev/nfs rootwait nfsroot=192.168.56.15:/srv/nfs/50900-h3-weston,vers=3 ip=dhcp' => bootcmd_vb 'setenv bootargs \${bootargs_vb};bootp 0x48080000 Image;tftp 0x48000000 \${dtb};run booti_cmd' => saveenv => run bootcmd_vb</pre>